학습 노트: vector_search 노드 — pgVector RAG 검색 패턴

작성일: 2026-05-24 관련 파일: `dev2/nodes/vector_search.py` (구현 예정), `dev2/db.py` (참고), `dev2/nodes/llm_judge.py` (다음 노드), `dev2/prompts/severity_judge.py` (similar_cases 소비처), `dev2/schema.py` 복습 예정: D+1, D+3, D+7 관련 **Task**: vector_search 노드 — CLI raw 이슈를 pgVector로 과거 유사 이슈 검색 → enriched 보강

---

0. 이 노드가 무슨 일을 하나요? (전체 그림 먼저)

비유: 신입 의사 옆의 베테랑 간호사

CLI 도구(pylint, bandit, semgrep)는 환자(코드)를 보고 "열이 나네요"라고 증상만 말합니다. 하지만 베테랑 간호사는 그 증상을 듣자마자 "아, 작년에 비슷한 증상으로 온 환자가 **3**명 있었는데 다 같은 병이었어요" 라며 과거 진료 기록(과거 PR 이슈)을 꺼내옵니다.

`vector_search` 노드가 바로 이 베테랑 간호사입니다. CLI 도구가 찾아낸 이슈 메시지를 받아서, 우리 팀의 과거 이슈 데이터베이스(pgVector)에서 의미가 비슷한 과거 사례를 찾아옵니다. 그리고 그 사례를 이슈 옆에 붙여줍니다(`similar_issues`).

이렇게 붙여진 과거 사례는 다음 노드인 `llm_judge`가 "과거에 이런 건 **Major**였으니 이번 것도 **Major**겠네" 라고 더 정확하게 판단하는 데 쓰입니다. 이게 RAG(검색 증강 생성)의 핵심입니다.

파이프라인에서의 위치

```
run_tools_node          →  vector_search_node       →  llm_judge_node
(CLI 도구 원본 결과)          (과거 유사 이슈 검색·주입)     (RAG context로 재판단)
static_raw / security_raw  → enriched 필드 채움      → enriched의 similar_issues 사용
```

- 입력: `state["static_raw"]`, `state["security_raw"]` (run_tools 노드가 채운 CLI 원본)
- 출력: `{"enriched": [...]}` — 각 이슈에 `similar_issues` 필드를 붙인 목록 (`Dev2State.enriched`, schema.py:38)
- 다음 노드: `llm_judge_node`가 `enriched`를 읽어 `issue["similar_issues"]`를 RAG context로 사용 (llm_judge.py:170)

> 핵심: `vector_search`는 검색만 합니다 (LLM 호출 안 함). LLM 판단은 `llm_judge`의 일입니다. 단일 책임 원칙(SRP) — "검색하는 사람"과 "판단하는 사람"을 나눠야 각각 따로 테스트할 수 있습니다.

개념 1. pgVector 코사인 유사도 검색

비유

도서관에서 책을 찾을 때, 책마다 "주제 좌표"가 적힌 스티커가 붙어 있다고 상상하세요. 내가 찾는 주제의 좌표를 알면, 좌표가 가장 가까운 책들을 순서대로 꺼낼 수 있습니다. pgVector의 `<=>` 연산자가 바로 "두 좌표 사이 거리"를 재는 자입니다.

왜 **pgVector**인가? **(팀 표준, CLAUDE.md §9)**

> ⚠️ **Milvus** 금지. 우리 팀 Vector DB는 pgVector입니다.

pgVector는 별도 Vector DB 서버(Milvus, Pinecone 등)를 띄우지 않고, 이미 쓰고 있는 **PostgreSQL**에 확장**(extension)**만 추가해서 벡터 검색을 하는 방식입니다. - 인프라가 하나로 통일됨 — PostgreSQL 하나만 운영하면 됨 (Docker Compose 단순화) - 메타데이터(트랜잭션, JOIN)와 벡터를 같은 DB에서 다룰 수 있음 - `scan_issues` 같은 일반 테이블과 같은 곳에 있으니 데이터 정합성 관리가 쉬움

`<=>` 연산자**:** 코사인 거리

```
코사인 거리 = 1 - 코사인 유사도

거리 0.0  → 유사도 1.0 (완전히 같은 의미)
거리 0.3  → 유사도 0.7 (꽤 비슷함)
거리 1.0  → 유사도 0.0 (전혀 관계 없음)
```

핵심 혼동 주의: `<=>`가 돌려주는 값은 거리입니다. 작을수록 비슷합니다. 우리가 원하는 "유사도"는 `1 - 거리`로 계산해야 합니다.

```sql
SELECT
    chunk_id,
    content,
    metadata,
    1 - (embedding <=> $1::vector) AS similarity   -- 거리를 유사도로 변환
FROM chunk_embeddings
ORDER BY embedding <=> $1::vector                   -- 거리가 작은 순(=비슷한 순)
LIMIT $2;                                           -- top-k개만
```

- `$1::vector` : 검색 기준 벡터(내 이슈 메시지의 임베딩)를 vector 타입으로 캐스팅
- `ORDER BY embedding <=> $1::vector` : 거리 오름차순 = 가장 비슷한 것부터
- `LIMIT $2` : 상위 k개만 (top-k)

> 다른 연산자도 있음: `<->`(L2/유클리드 거리), `<#>`(내적). 우리는 의미 유사도용으로 코사인**(`<=>`)**을 씁니다. 텍스트 임베딩은 방향(의미)이 중요하고 크기는 덜 중요하기 때문입니다.

HNSW 인덱스: 왜 필요한가?

브루트포스(인덱스 없음): 검색할 때마다 테이블의 모든 행과 일일이 거리 계산합니다. 10만 개 이슈면 10만 번 계산. 정확하지만 느립니다 (O(n)).

HNSW(Hierarchical Navigable Small World): 미리 벡터들을 "고속도로 + 골목길" 같은 다층 그래프로 연결해 둡니다. 검색할 때 고속도로로 대략 가까운 동네까지 빠르게 이동한 뒤 골목길에서 정밀 탐색합니다. 전수조사 없이 근사적으로 가장 가까운 것을 찾습니다 (대략 O(log n)).

```
비유:
  브루트포스 = 서울 모든 집을 한 채씩 방문해서 목적지 찾기
  HNSW       = 고속도로 타고 강남까지 → 역삼동 → 골목 → 목적지
```

- 트레이드오프: HNSW는 100% 정확하지는 않음(근사). 하지만 보통 95%+ 재현율로 수십~수백 배 빠름. 우리 용도(과거 유사 사례 참고)에는 충분합니다.
- 인덱스 생성 예시 (코사인용 `vector_cosine_ops` 지정 필수):

```
CREATE INDEX idx_chunk_embeddings_hnsw
    ON chunk_embeddings
    USING hnsw (embedding vector_cosine_ops);
```

> ⚠️ 인덱스를 만들 때의 연산자 클래스(`vector_cosine_ops`)와 쿼리 연산자(`<=>`)가 짝이 맞아야 인덱스가 실제로 사용됩니다. L2용 인덱스를 만들고 `<=>`로 검색하면 인덱스를 안 타고 브루트포스로 돌아갑니다.

검색 결과 0건 처리 (CLAUDE.md §12)

처음 발견되는 패턴이면 과거 사례가 없을 수 있습니다. 이때 예외를 던지면 안 됩니다. 빈 리스트를 반환하고 그냥 진행합니다.

```
if not rows:
    logger.warning("vector_search: 유사 이슈 0건 — '%s'", message[:50])
    return []   # 예외 던지지 않음. similar_issues=[]로 다음 노드 진행
```

이 빈 리스트는 `severity_judge.py:62`에서 "유사 사례 없음 (처음 발견된 패턴)"으로 자연스럽게 표시됩니다.

개념 2. asyncpg — PostgreSQL 비동기 클라이언트

비유

psycopg2(동기)는 창구가 하나인 은행입니다. 한 손님(쿼리)이 일을 끝낼 때까지 뒤의 모든 손님이 줄 서서 기다립니다. asyncpg(비동기)는 번호표 시스템입니다. 한 손님이 서류 작성

(DB 응답 대기) 하는 동안 직원은 다른 손님을 처리합니다. 기다리는 시간을 낭비하지 않습니다.

왜 **psycopg2** 대신 **asyncpg**인가?
우리 노드들은 전부 `async def` + `asyncio.gather`로 돌아갑니다 (run_tools.py, llm_judge.py 참고). 이 비동기 흐름 안에서 동기 라이브러리**(psycopg2)**를 쓰면 이벤트 루프 전체가 멈춥니다. DB 응답을 기다리는 동안 다른 이슈의 검색도 다 멈춰버립니다.

asyncpg는 `await`로 동작하므로, 한 이슈가 DB 응답을 기다리는 동안 다른 이슈 검색을 동시에 진행할 수 있습니다. asyncio와 궁합이 맞습니다.

> 참고: 현재 `dev2/db.py`는 SQLAlchemy + `asyncpg` 드라이버(`postgresql+asyncpg://`)를 씁니다. vector_search도 같은 흐름과 일관되게 비동기로 작성합니다. 아래는 asyncpg를 직접 쓰는 패턴이며, 팀이 SQLAlchemy로 통일하면 `text()` + `AsyncSessionLocal` 방식으로도 동일하게 구현 가능합니다.

`connect()` vs `create_pool()` 차이

	`asyncpg.connect()`	`asyncpg.create_pool()`
비유	쿼리마다 은행 문 열고 닫기	미리 직원 N명 대기시켜 둠
동작	매번 새 TCP 연결 생성·종료	연결 N개를 미리 만들어 재사용
비용	연결 생성 비용이 매번 발생 (느림)	연결 재사용으로 빠름
적합	일회성 스크립트, 마이그레이션	서버/노드처럼 반복 호출되는 곳

우리 vector_search 노드는 **PR**마다 여러 이슈를 반복 검색하므로 풀**(pool)**이 맞습니다. 다만 풀은 노드 안에서 매번 만들지 말고 앱 시작 시 한 번 만들어 외부에서 주입해야 합니다 (개념 4 참고).

`fetch` 패턴 + 벡터 파라미터 전달
asyncpg는 `$1, $2` 형태의 위치 파라미터를 씁니다 (`%s` 아님). 벡터는 리스트를 그대로 못 넣고, 문자열 **`'[...]'`**로 바꿔서 **`::vector`**로 캐스팅해야 합니다.

```
import asyncpg

async def search_similar(pool: asyncpg.Pool, query_vec: list[float], k: int) ->
list[dict]:
    # ⚠ 핵심: list[float] → "[0.1,0.2,...]" 문자열로 변환 후 ::vector 캐스팅
```

```
    vec_str = "[" + ",".join(str(x) for x in query_vec) + "]"

    sql = """
        SELECT chunk_id, content, metadata,
               1 - (embedding <=> $1::vector) AS similarity
        FROM chunk_embeddings
        ORDER BY embedding <=> $1::vector
        LIMIT $2
    """
    # 파라미터 바인딩만 사용 — f-string으로 SQL 조합 금지 (CLAUDE.md §6 보안 규칙)
    rows = await pool.fetch(sql, vec_str, k)   # asyncpg.Record 리스트 반환
    return [dict(r) for r in rows]             # dict로 변환해 다루기 쉽게
```

- `await pool.fetch(sql, *args)` : SELECT 결과 여러 행을 `Record` 리스트로 반환
- `pool.fetchrow(...)` : 한 행만, `pool.fetchval(...)` : 단일 값만
- `Record`는 dict처럼 `r["chunk_id"]`로 접근 가능. `dict(r)`로 일반 dict 변환 가능

> ⚠️ **f-string으로 SQL을 조합하지 마세요** (CLAUDE.md §6). 반드시 `$1, $2` 파라미터 바인딩. 벡터 문자열도 SQL 본문이 아니라 파라미터 값으로 전달합니다(`$1::vector`). 이렇게 해야 SQL 인젝션이 차단됩니다.

개념 3. 임베딩 생성 패턴 (Mock 전환 포함)

비유

임베딩은 글을 좌표로 바꾸는 번역기입니다. "SQL 인젝션 취약점"이라는 문장을 [0.12, −0.34, ...] 같은 숫자 좌표로 번역합니다. 의미가 비슷한 문장은 좌표도 가까이 찍힙니다.

흐름: 이슈 메시지 → 벡터

```
issue["message"] = "사용자 입력이 SQL 쿼리에 직접 삽입됨"
        ↓ embed_text()
[0.12, -0.34, 0.89, ...]   ← 길이 = PGVECTOR_DIMENSION
        ↓ ::vector 캐스팅 후 pgVector 검색
과거 유사 이슈 목록
```

`USE_MOCK_LLM=true` 시 Mock 임베딩 (CLAUDE.md §6, §10-3)

개발 중에는 실제 임베딩 API를 부르면 안 됩니다(비용·속도·테스트 독립성). `llm_judge.py`가 Mock LLM을 쓰듯, vector_search도 **Mock** 임베딩을 반환합니다. 호출 측은 Mock인지 실제인지 구분하지 않아도 됩니다 — 환경변수만 바꾸면 됩니다.

```
import os, hashlib

PGVECTOR_DIMENSION = int(os.getenv("PGVECTOR_DIMENSION", "4096"))  # CLAUDE.md §8: 4096 또는 768
```

```python
def _mock_embedding(text: str) -> list[float]:
    """
    결정적(deterministic) Mock 임베딩 — 같은 텍스트는 항상 같은 벡터.
    해시 기반이라 테스트에서 재현 가능. 실제 의미는 없지만 차원/타입은 동일.
    """
    seed = int(hashlib.sha256(text.encode()).hexdigest(), 16)
    # 0~1 사이 값을 차원 수만큼 생성 (실제 임베딩과 길이/타입 동일)
    return [((seed >> (i % 64)) & 0xFFFF) / 65535.0 for i in
range(PGVECTOR_DIMENSION)]

async def embed_text(text: str) -> list[float]:
    use_mock = os.getenv("USE_MOCK_LLM", "true").lower() == "true"
    if use_mock:
        return _mock_embedding(text)       # 실제 API 호출 없음
    # 실제: 팀 결정 임베딩 모델 (Upstage 4096 / Ollama 768) 호출
    # vec = await call_real_embedding_api(text)
    # assert len(vec) == PGVECTOR_DIMENSION  # 차원 검증
    # return vec
    raise NotImplementedError("실제 임베딩 연동은 Week 5 통합 시")
```

차원 통일 **(CLAUDE.md §8)** — 가장 흔한 첫 버그

임베딩 길이(차원)는 **3**곳이 반드시 일치해야 합니다: 1. `.env`의 `PGVECTOR_DIMENSION` (4096=Upstage / 768=Ollama) 2. DDL의 `vector(4096)` 또는 `vector(768)` 3. seed INSERT의 `::vector(4096)` 또는 `::vector(768)`

하나라도 다르면 첫 **INSERT**/검색에서 **"차원 불일치(dimension mismatch)"** 오류가 납니다. Mock 임베딩도 반드시 `PGVECTOR_DIMENSION` 길이로 생성해야 실제 전환 시 코드 수정이 없습니다.

개념 4. LangGraph 노드에서의 DB 연결 패턴

비유

노드 함수 안에서 매번 DB에 연결하는 것은 요리할 때마다 새 부엌을 짓는 것과 같습니다. 비효율적이고, 테스트할 때 진짜 부엌(DB)이 없으면 요리(테스트)를 못 합니다. 대신 이미 지어진 부엌(연결 풀)을 외부에서 빌려 쓰면 빠르고, 테스트 때는 장난감 부엌(Mock 풀)을 빌려주면 됩니다.

노드 내부 연결 **vs** 외부 주입(권장)

```python
# ❌ 안티패턴: 노드가 직접 연결 생성 (매번 비용 발생 + 테스트 시 실제 DB 필요)
async def vector_search_node(state):
    conn = await asyncpg.connect(os.getenv("POSTGRES_URL"))  # 매 호출마다 연결
    ...
```

```python
# ✅ 권장: 풀을 외부에서 주입 (functools.partial로 LangGraph에 바인딩)
async def vector_search_node(state: Dev2State, pool: asyncpg.Pool) -> dict:
    ...
    rows = await pool.fetch(sql, vec_str, k)
    ...

# 그래프 빌드 시 풀을 미리 만들어 주입
from functools import partial
pool = await asyncpg.create_pool(POSTGRES_URL)   # 앱 시작 시 1회
graph.add_node("vector_search", partial(vector_search_node, pool=pool))
```

왜 **POSTGRES_URL**을 환경변수로 외부화하나?

- 환경별 전환: 로컬은 `localhost:5432`, Docker는 `postgres:5432`, CI는 테스트 DB — 코드 변경 없이 **.env**만 바꾸면 됩니다 (CLAUDE.md §10-3).
- 비밀 노출 방지: 비밀번호를 코드에 하드코딩하면 git에 올라갑니다. 환경변수는 코드 밖에 둡니다.
- 현재 표준: `dev2/db.py:7`이 이미 `os.getenv("DATABASE_URL", ...)` 패턴을 씁니다. 같은 규칙을 따르세요.

테스트 시 **DB**를 **Mock**으로 교체

실제 DB 없이 단위 테스트가 통과해야 합니다 (CLAUDE.md §10-5). 풀을 외부 주입하면 가짜 풀을 넣기 쉽습니다.

```python
from unittest.mock import AsyncMock, MagicMock

async def test_vector_search_returns_similar_issues():
    # 가짜 풀: fetch가 미리 정한 행을 반환
    fake_pool = MagicMock()
    fake_pool.fetch = AsyncMock(return_value=[
        {"chunk_id": "c-1", "content": "N+1 쿼리...", "metadata": {"pr_id":
"PR-001"}, "similarity": 0.82},
    ])
    state = {"static_raw": [{"message": "for 루프 안 쿼리", "severity": "Major"}],
"security_raw": []}

    result = await vector_search_node(state, pool=fake_pool)

    assert result["enriched"][0]["similar_issues"][0]["similarity"] == 0.82  # 실
제 DB 없이 통과

async def test_vector_search_empty_result():
    fake_pool = MagicMock()
    fake_pool.fetch = AsyncMock(return_value=[])      # 0건
    state = {"static_raw": [{"message": "처음 보는 패턴", "severity": "Minor"}],
"security_raw": []}
    result = await vector_search_node(state, pool=fake_pool)
```

```
    assert result["enriched"][0]["similar_issues"] == []   # 예외 없이 빈 리스트
(CLAUDE.md §12)
```

개념 5. RAG Retrieve 패턴 (전체 조립)

비유

RAG는 오픈북 시험입니다. LLM이 머릿속 지식만으로 답하지 않고(closed-book), 우리 팀의 과거 이슈 노트(pgVector)를 펼쳐보고(Retrieve) 답합니다(Generate). 그래서 "우리 팀 맥락"에 맞는 정확한 판단이 나옵니다.

Retrieve 흐름

```
1. issue["message"]            (CLI 도구가 찾은 이슈 메시지)
2.   → embed_text(message)     (임베딩: 텍스트 → 벡터, 개념 3)
3.   → pgVector <=> 검색       (코사인 유사도 top-k, 개념 1)
4.   → threshold 필터 (유사도 ≥ 0.7)
5.   → issue["similar_issues"] 주입
6.   → enriched에 담아 다음 노드(llm_judge)로
```

top-k를 **similar_issues**에 주입 — **severity_judge**가 기대하는 형식

severity_judge.py:50이 기대하는 각 항목 구조: {"pr_id": str, "similarity": float, "message": str}. vector_search는 검색 결과를 정확히 이 형식으로 변환해 넣어야 합니다.

```
SIMILARITY_THRESHOLD = 0.7   # 이 미만은 버림
TOP_K = 3

async def vector_search_node(state: Dev2State, pool: asyncpg.Pool) -> dict:
    raw_issues = state.get("static_raw", []) + state.get("security_raw", [])
    if not raw_issues:
        logger.warning("vector_search_node: raw 이슈 없음 — enriched 빈 리스트")
        return {"enriched": []}

    enriched: list[dict] = []
    for issue in raw_issues:
        message = issue.get("message", "")
        query_vec = await embed_text(message)              # 개념 3
        vec_str = "[" + ",".join(str(x) for x in query_vec) + "]"

        rows = await pool.fetch("""
            SELECT chunk_id, content, metadata,
                   1 - (embedding <=> $1::vector) AS similarity
            FROM chunk_embeddings
            ORDER BY embedding <=> $1::vector
            LIMIT $2
        """, vec_str, TOP_K)                               # 개념 1, 2
```

```
        # threshold 필터 + severity_judge가 기대하는 형식으로 변환
        similar = [
            {
                "pr_id": dict(r).get("metadata", {}).get("pr_id", "unknown"),
                "similarity": round(r["similarity"], 4),
                "message": r["content"],
            }
            for r in rows
            if r["similarity"] >= SIMILARITY_THRESHOLD       # 개념 5: 0.7 미만 버
림
        ]

        if not similar:
            logger.info("vector_search_node: '%s' 유사 이슈 0건(threshold)",
message[:40])

        enriched.append({**issue, "similar_issues": similar})  # 원본 보존 + 필드
추가

    logger.info("vector_search_node 완료: %d개 이슈 보강", len(enriched))
    return {"enriched": enriched}
```

왜 유사도 **0.7** 이상만 쓰나**?**

- 노이즈 차단: top-k는 무조건 k개를 돌려줍니다. 정말 비슷한 게 없어도 "그나마 덜 다른 것" k개를 줍니다. 유사도 0.3짜리(거의 무관한) 사례를 LLM에 넣으면 오히려 판단을 오염시킵니다.
- **RAG** 품질 = 검색 품질: 쓰레기를 넣으면 쓰레기가 나옵니다(garbage in, garbage out). threshold는 "충분히 관련 있는 것만" 거르는 안전장치입니다.
- **0.7**은 출발점: 절대 정답값이 아닙니다. 실제 데이터로 검색 품질을 보며 0.65~0.8 사이에서 튜닝합니다.

3rd-Project-sub2 프로젝트에서

파일: `dev2/nodes/vector_search.py` (구현 예정) 역할: `run_tools_node`가 만든 CLI raw 이슈(`static_raw`/`security_raw`)를 받아, 각 이슈 메시지를 임베딩 → pgVector 코사인 검색으로 과거 유사 이슈를 찾고, `similar_issues` 필드를 붙여 `enriched`로 반환한다. 이 `enriched`는 다음 노드 `llm_judge_node`가 RAG context로 사용한다.

연결 관계: - 앞 노드: `dev2/nodes/run_tools.py` → `static_raw`, `security_raw` 제공 - 뒤 노드: `dev2/nodes/llm_judge.py:170` → `issue["similar_issues"]` 소비 - 형식 계약: `dev2/prompts/severity_judge.py:50` → `{"pr_id", "similarity", "message"}` 기대 - DB 패턴 참고: `dev2/db.py` → 비동기 + `os.getenv` 외부화 + 파라미터 바인딩 - State 정의: `dev2/schema.py:38` → `enriched: list[dict]`

연관 개념

- 임베딩**(Embedding)** → 텍스트를 의미 좌표(숫자 배열)로 바꾸는 것. `docs/learning/vector/vectordb.md` 참고
- **HNSW** 인덱스 → 전수조사 없이 빠르게 근사 최근접 이웃을 찾는 그래프 인덱스
- 코사인 거리 **`<=>`** → 1 - 코사인 유사도. 작을수록 비슷함
- **asyncpg** 풀 → 미리 만든 연결 N개를 재사용하는 비동기 PostgreSQL 클라이언트
- **RAG** → 검색(Retrieve) + 생성(Generate). 외부 지식을 찾아 LLM에 넣어주는 패턴
- **Mock** 전환 → `USE_MOCK_LLM`으로 실제 API 없이 개발/테스트 (llm_judge.py와 동일 철학)

복습 퀴즈 **(3**문항**)**

1. **(**이해 확인**)** pgVector에서 `embedding <=> $1::vector`가 0.3을 반환했습니다. 이 두 벡터의 코사인 유사도는 얼마이고, "비슷한 편"인가요 "다른 편"인가요?

2. **(**적용**)** vector_search 노드가 어떤 이슈에 대해 과거 사례를 한 건도 못 찾았습니다(0건). CLAUDE.md §12에 따르면 이 노드는 무엇을 반환해야 하며, 다음 노드(llm_judge)와 severity_judge는 이를 어떻게 처리하나요?

3. **(**비교**)** 10만 개 이슈가 쌓인 운영 환경에서 HNSW 인덱스를 만들지 않고 `<=>` 검색을 하면 무슨 일이 생기나요? HNSW를 만들 때 `vector_cosine_ops`를 빠뜨리면 어떤 문제가 발생하나요?